COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d

Week 5

Writing MapReduce Program -III

DEVELOPMENT TIPS AND TECHNIQUES

Strategies for Debugging MapReduce Code

Debugging MapReduce code is not easy.

- Each instance of a Mapper runs as a separate task (Often on a different machine).
- Difficult to attach a debugger to the process.
- Difficult to catch “edge cases.”

Very large volumes of data means that unexpected input is likely to appear

- Code which expects all data to be well-formed is likely to fail.

Debugging Tips

- With sampled small-scale data.

Code defensively

Start small, build incrementally

Make as much of your code as possible Hadoop-agnostic

Write unit tests

Test locally whenever possible

Test in pseudo-distributed mode

Finally, test on the cluster

- Ensure that input data is in the expected format.
- Expect things to go wrong and catch exceptions.

- Make it easier to test.

Testing Strategies

When testing in pseudo-distributed mode, ensure that you are testing with a similar environment to that on the real cluster.

- Same amount of RAM allocated to the task JVMs.
- Same Hadoop version.
- Same Java version.
- Same third-party library versions.

Testing Locally Using LocalJobRunner

Hadoop can run Mapreduce in a single, local process.

- Does not require any Hadoop daemons to be running.
- Uses the local file system instead of HDFS.
- Known as LocalJobRunner mode.

This is a very useful way for quickly testing incremental changes to code.

Testing via LocalJobRunner

- To run in LocalJobRunner mode, add the following lines to the driver code:

```
Configuration conf = new Configuration();
conf.set("mapred.job.tracker", "local");
conf.set("fs.default.name","file:///");
```

In Hadoop 1.x only.

```
Configuration conf = new Configuration();
conf.set("mapreduce.framework.name","local");
conf.set("fs.defalutFS", "file:///");
Job job = Job.getInstance(getConf(), "Code Local Job Runner Word Count");
```

In Hadoop 2.x and above.

- Or, set these options on the command line if your driver uses ToolRunner
  - -fs is equivalent to -D *fs.default.name*
  - -jt is equivalent to -D *maprep.job.tracker*

In Hadoop 1.x only.

hadoop jar *yourjar.jar YourDriver* -fs=file:/// -jt=local *Input Output*

```
hadoop jar ljrwc.jar stubs.LocalWordCount -fs=file:/// -jt=local shakespeare.tar.gz cmljrwc_res
```

Limitations of LocalJobRunner Mode

- Distributed Cache does NOT work.
- The job can only specify a single Reducer.
- Some “beginner” mistakes may not be caught. For example, attempting to share data between Mappers will work (which it should not be valid in the real situation) because the code is running in a single JVM.

LocalJobRunner Mode in Eclipse

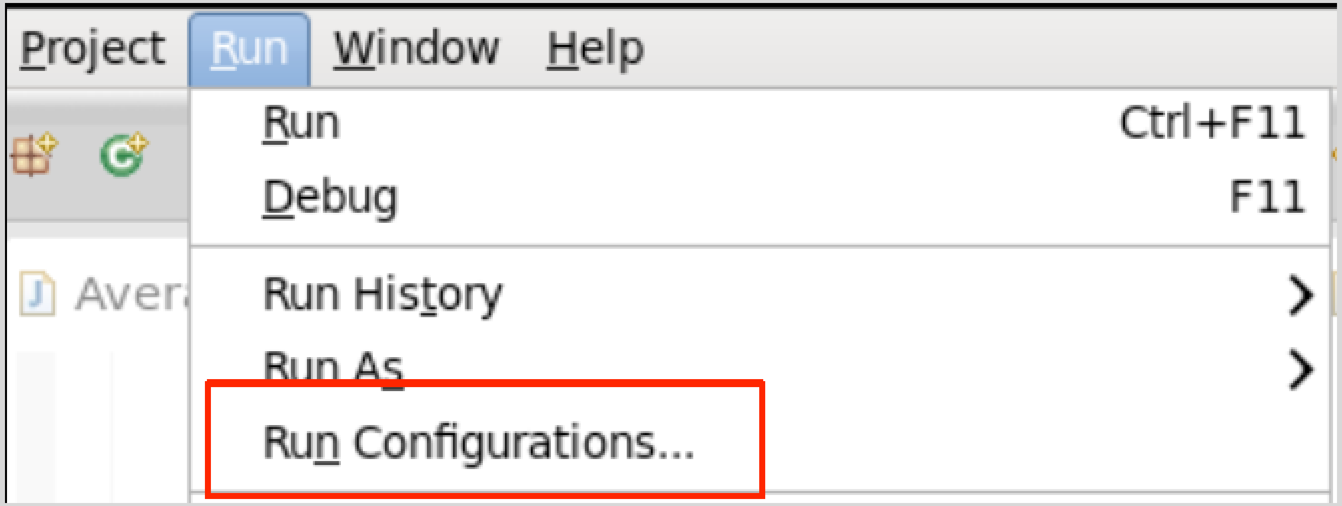
In the VM we used in the lab, the Eclipse can also run Hadoop code in LocalJobRunner mode from within the IDE.

- This is Hadoop’s default behaviour when no configuration is provided.

This allows rapid development iterations. (Agile programming)

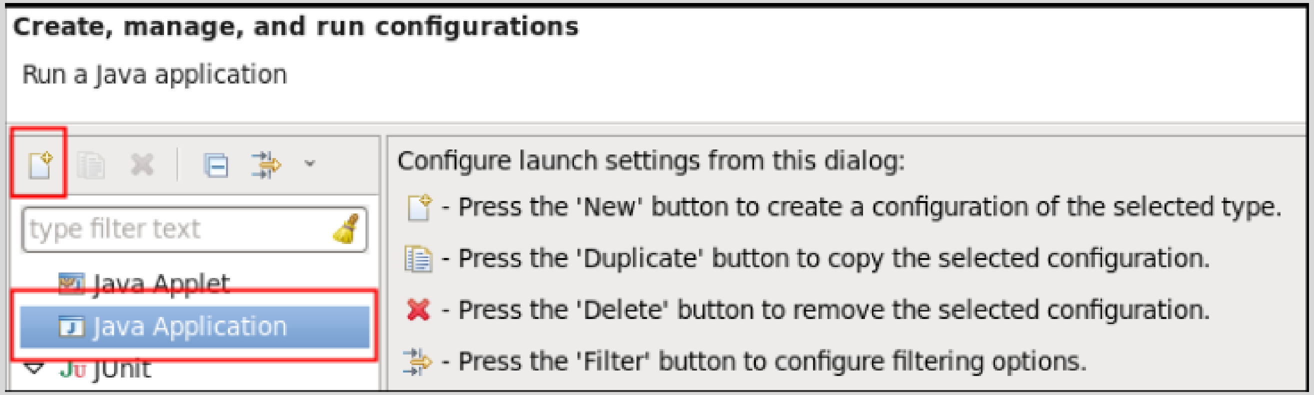
LocalJobRunner Mode in Eclipse

- **Find the corresponding setting in Eclipse**

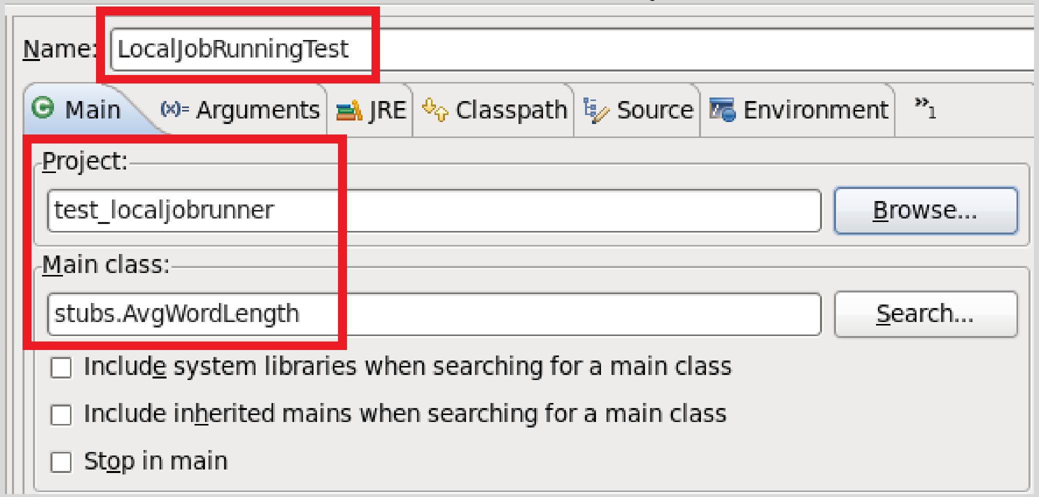


LocalJobRunner Mode in Eclipse

- **Select Java Application and then click on the "New" button.**

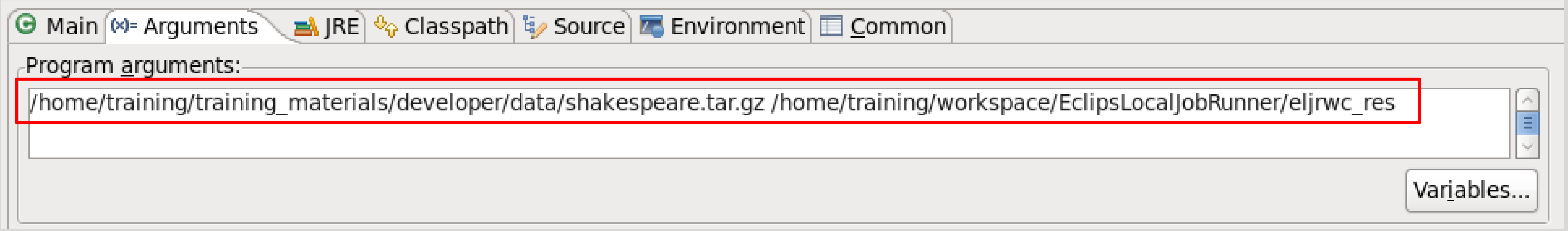


- **Verify the Project and Main Class fields are pre-filled correctly.**

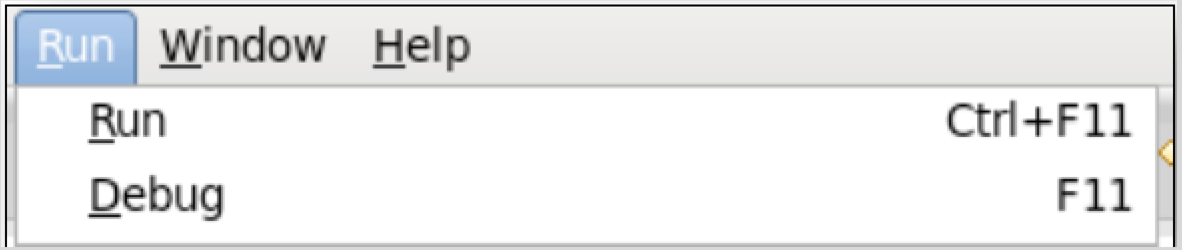


LocalJobRunner Mode in Eclipse

- Specify values in the Arguments tab.
  - Local input/output file
  - Any configuration options needed when your job runs.

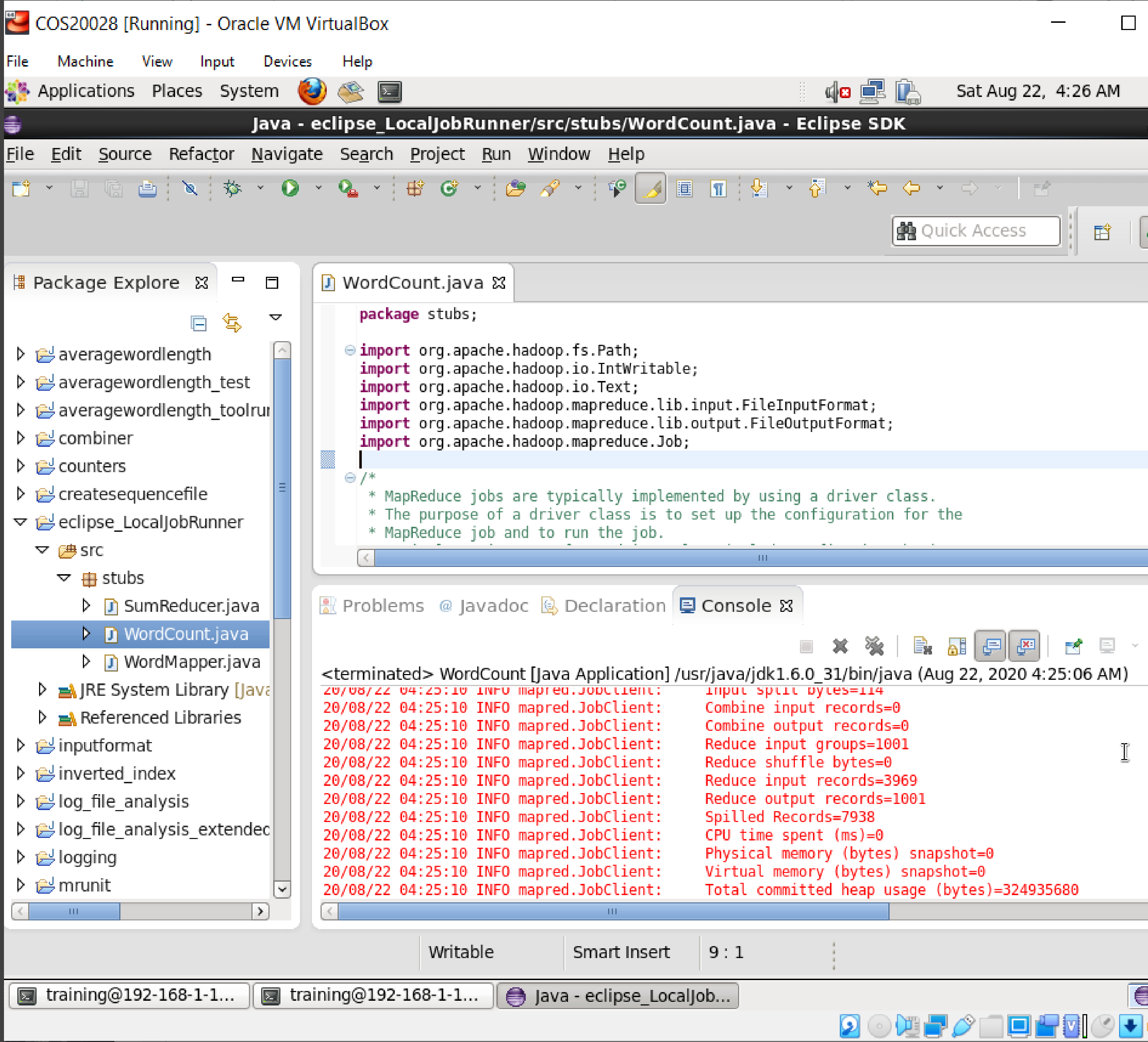


- Define breakpoints if desired.
- Execute the application in run mode or debug mode.



LOCALJOBRUNNER MODE IN ECLIPSE

Review output in the Eclipse console window



WRITING AND VIEWING LOG FILES

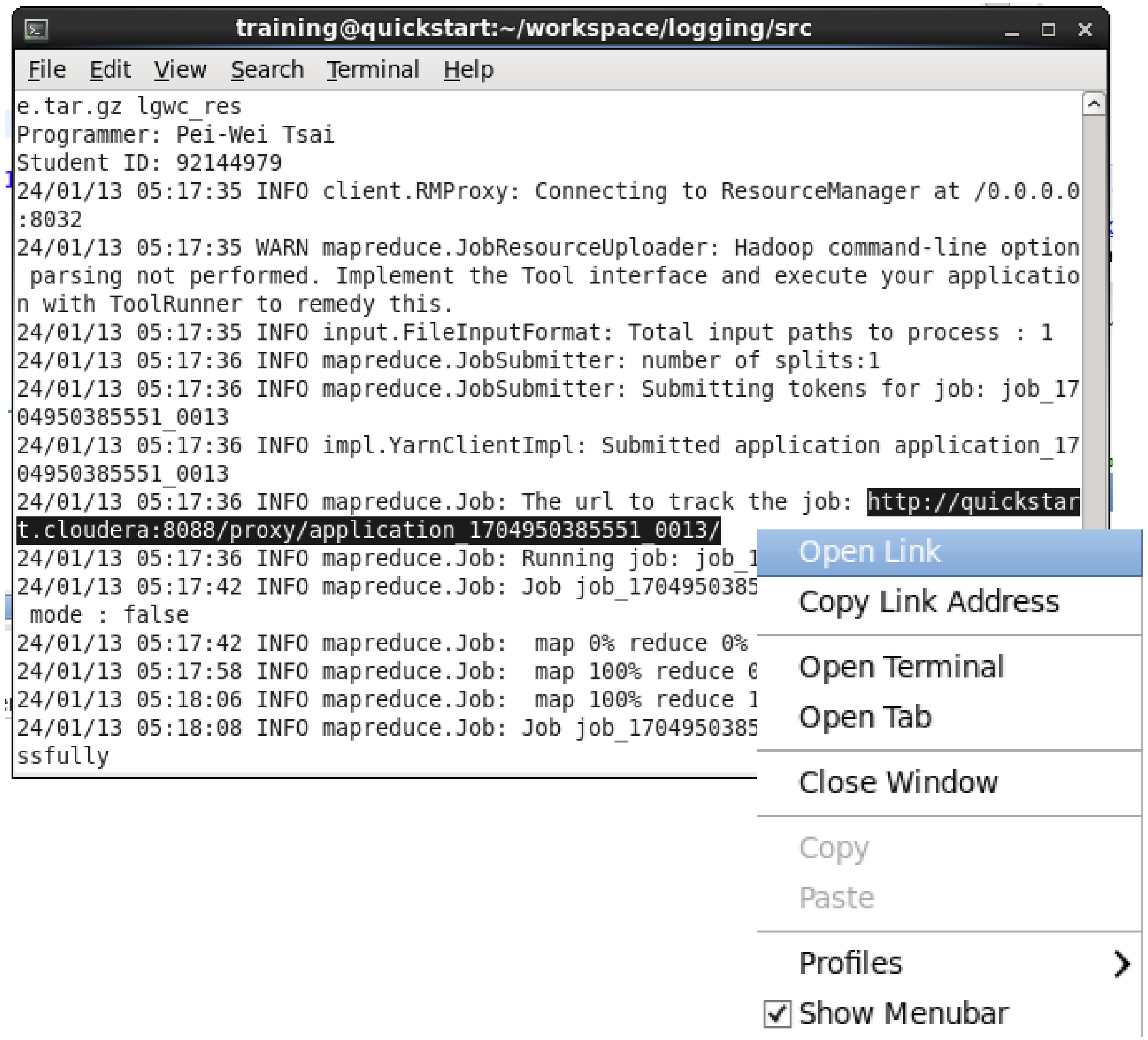
Using *stdout* and *stderr*

Tried-and-true debugging technique: write to *stdout* or *stderr*

- If running in LocalJobRunner mode, you will see the results of *System.err.println()*

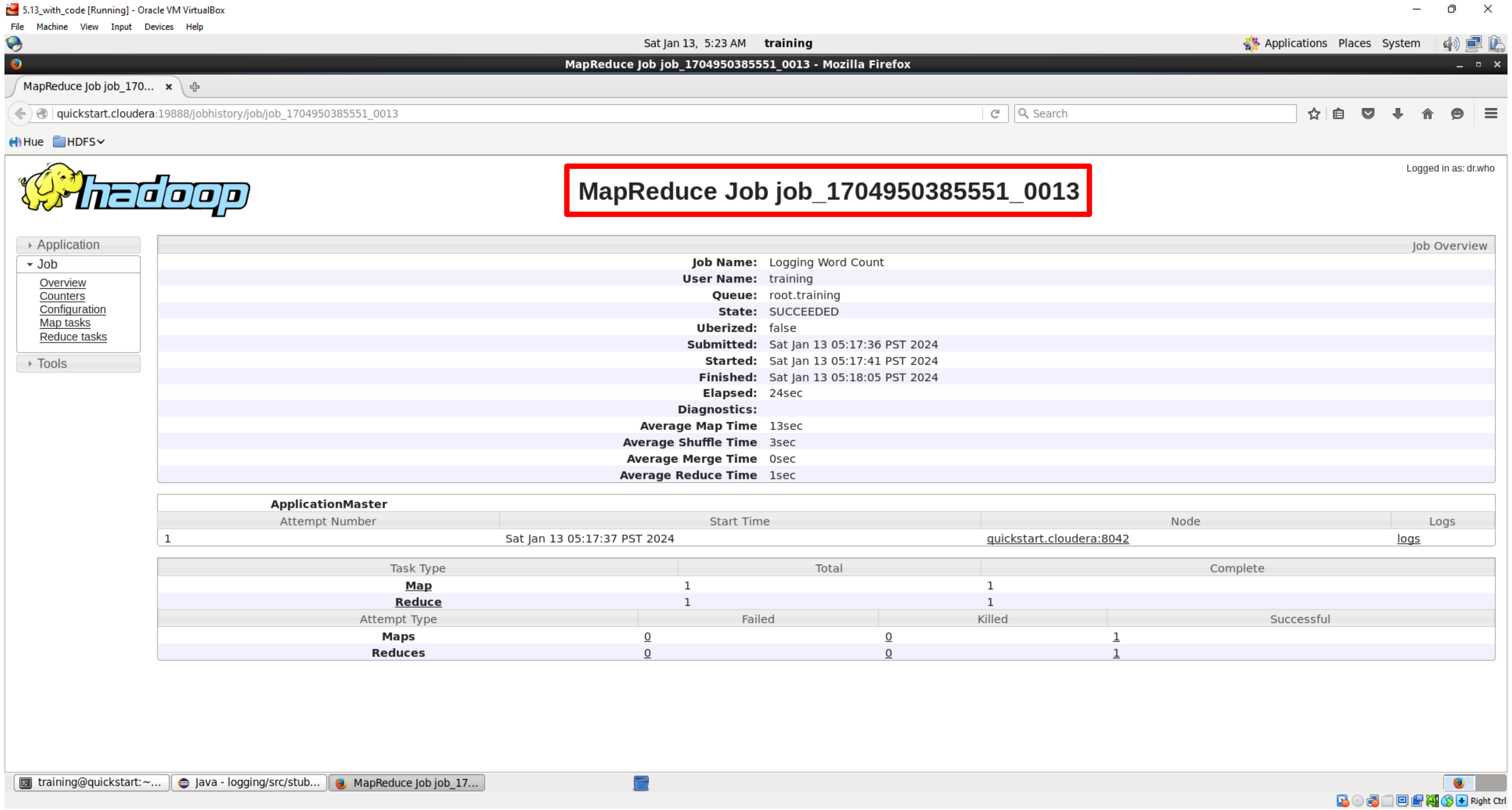
If running on a cluster, that output will not appear on your console

- Output is visible via Hadoop's Web UI.



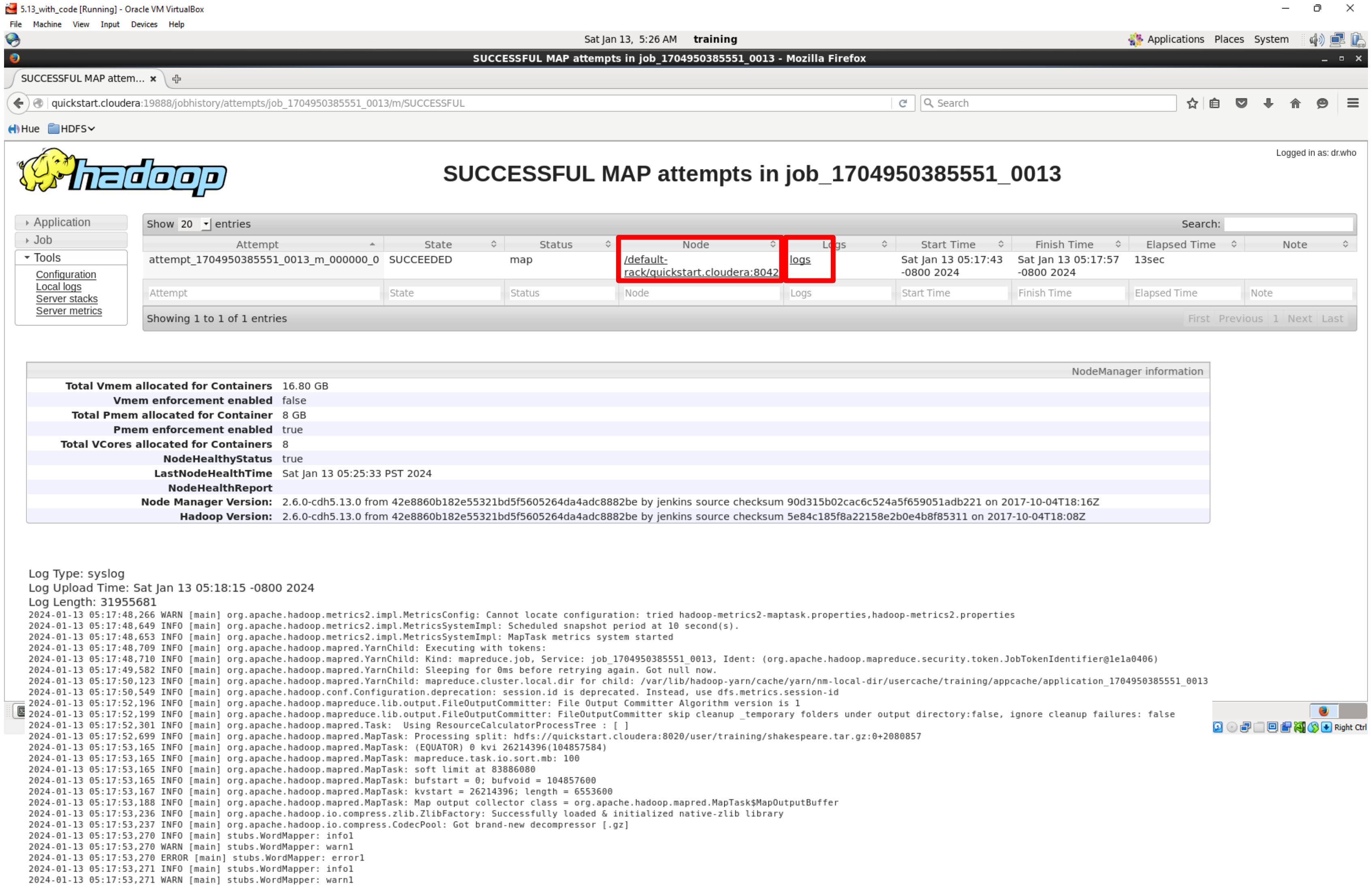
Hadoop Yarn Job Tracking

- Hadoop Yarn in an integrated platform, which provides the job tracking function.
- When executing your job, you will see a URL pops-up in the displayed information.
- Right-click the URL and click “Open Link” to see the tracking result.



LOGFILE VIA WEB UI

LOGFILE VIA WEB UI



Why Logging is better than Printing?

***println* statements rapidly become awkward**

- Turning them on and off in your code is tedious, and leads to errors.

Logging provides much finer-grained control over:

- What gets logged.
- When something gets logged.
- How something is logged.

log4j Library

- Hadoop uses *log4j* to generate all its log files.
- Your Mappers and Reducers can also use *log4j.*
  - All the initialisation is handled for you by Hadoop.
- Add the *log4j.jar-<version>* file from your CDH distribution to your classpath when you reference the *log4j* classes.

log4j Library

Simply send strings to loggers tagged with severity levels

- LOGGER.trace("*message*");
- LOGGER.debug("*message*");
- LOGGER.info("*message*");
- LOGGER.warn("*message*");
- LOGGER.error("*message*");

AvgWordLength.java | LetterMapper.java

```java
package stubs;
import java.io.IOException;

import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import org.apache.log4j.LogManager;
import org.apache.log4j.Logger;

public class LetterMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {

  private static final Logger LOGGER = LogManager.getLogger(LetterMapper.class.getName());

  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
```

USING *LOG4J* TO STORE LOGS

AvgWordLength.java | *LetterMapper.java

```java
@Override
public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

      String tmp =  context.getConfiguration().get("caseSensitive");
      Boolean caseSensitivie = (tmp.equalsIgnoreCase("true")? true : false);

      String line = (caseSensitivie? value.toString() : value.toString().toLowerCase());


      for (String word : line.split("\\W+")) {
        if (word.length() > 0) {
          LOGGER.info(word.substring(0,1) + " --> " + word.length());
          context.write(new Text(word.substring(0, 1)), new DoubleWritable(word.length()));
        }
      }

}
}
```

USING *LOG4J* TO STORE LOGS

USING *LOG4J* TO STORE LOGS

- Application
  - About
  - Jobs
- Tools

Log Type: stderr
Log Upload Time: Sat Jan 13 05:49:00 -0800 2024
Log Length: 0

Log Type: stdout
Log Upload Time: Sat Jan 13 05:49:00 -0800 2024
Log Length: 0

Log Type: syslog
Log Upload Time: Sat Jan 13 05:49:00 -0800 2024
Log Length: 62366633
Showing 4096 bytes of 62366633 total. Click here for the full log.

```
tterMapper: t --> 3
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: m --> 6
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: h --> 5
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: m --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: w --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: s --> 5
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: h --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: b --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: m --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: I --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: s --> 6
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: b --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: n --> 3
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: b --> 3
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: t --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: m --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: f --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: i --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: t --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: S --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: g --> 6
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: g --> 5
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: o --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: W --> 10
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: w --> 5
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: h --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: T --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: t --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: I --> 1
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: l --> 2
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: s --> 5
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: a --> 3
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: s --> 4
2024-01-13 05:48:40,095 INFO [main] stubs.LetterMapper: a --> 5
```

```java
package stubs;
import java.io.IOException;

import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.log4j.Level;
import org.apache.log4j.LogManager;
import org.apache.log4j.Logger;

public class LetterMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

  private static final Logger LOGGER = LogManager.getLogger(LetterMapper.class.getName());

  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {

      LOGGER.setLevel(Level.ALL);
      //LOGGER.setLevel(Level.DEBUG);
      //LOGGER.setLevel(Level.ERROR);
      //LOGGER.setLevel(Level.FATAL);
      //LOGGER.setLevel(Level.INFO);
      //LOGGER.setLevel(Level.TRACE);
      //LOGGER.setLevel(Level.WARN);


      String line = value.toString();

      for (String word : line.split("\\W+")) {
        if (word.length() > 0) {
          LOGGER.debug(word.substring(0,1) + " --> " + word.length());
          context.write(new Text( word.substring(0,1)), new IntWritable(word.length()));
        }
      }
  }
}
```

Avoiding Expensive Operations

- By sending the parameter via the ToolRunner, we can decide whether display the debugging information without modifying the code.

```java
package stubs;
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.log4j.Level;
import org.apache.log4j.LogManager;
import org.apache.log4j.Logger;

public class AverageReducer extends Reducer<Text, IntWritable, Text, DoubleWritable> {

    private static final Logger LOGGER = LogManager.getLogger(AverageReducer.class.getName());

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        LOGGER.setLevel(Level.ALL);
        //LOGGER.setLevel(Level.DEBUG);
        //LOGGER.setLevel(Level.ERROR);
        //LOGGER.setLevel(Level.FATAL);
        //LOGGER.setLevel(Level.INFO);
        //LOGGER.setLevel(Level.TRACE);
        //LOGGER.setLevel(Level.WARN);

        double wordlength = 0.0;
        double cnt = 0.0;

        for (IntWritable value : values) {
            wordlength += Double.valueOf(value.get());
            cnt += 1.0;

            LOGGER.warn(key + " --(Acc.)--> " + wordlength);
        }
        if(cnt > 0.0) {
            wordlength = wordlength / cnt;
        }

        context.write(key, new DoubleWritable(wordlength));
    }
}
```

AVOIDING EXPENSIVE OPERATIONS

Put a logger in the Reducer for outputting information.

CHECK THE LOG CONTENT

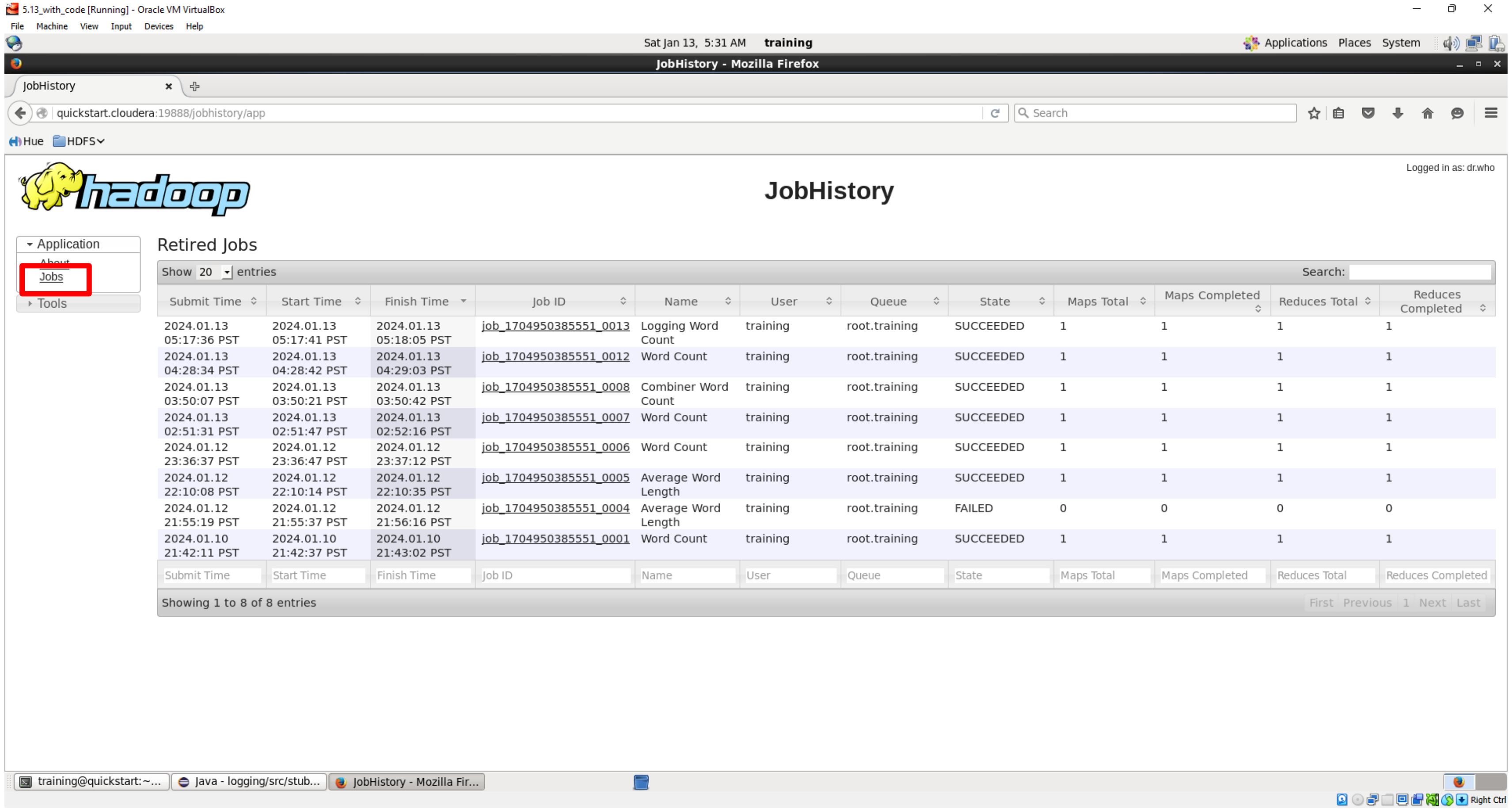
Find Your Job ID

training@quickstart:~/workspace/logging/src

File Edit View Search Terminal Help

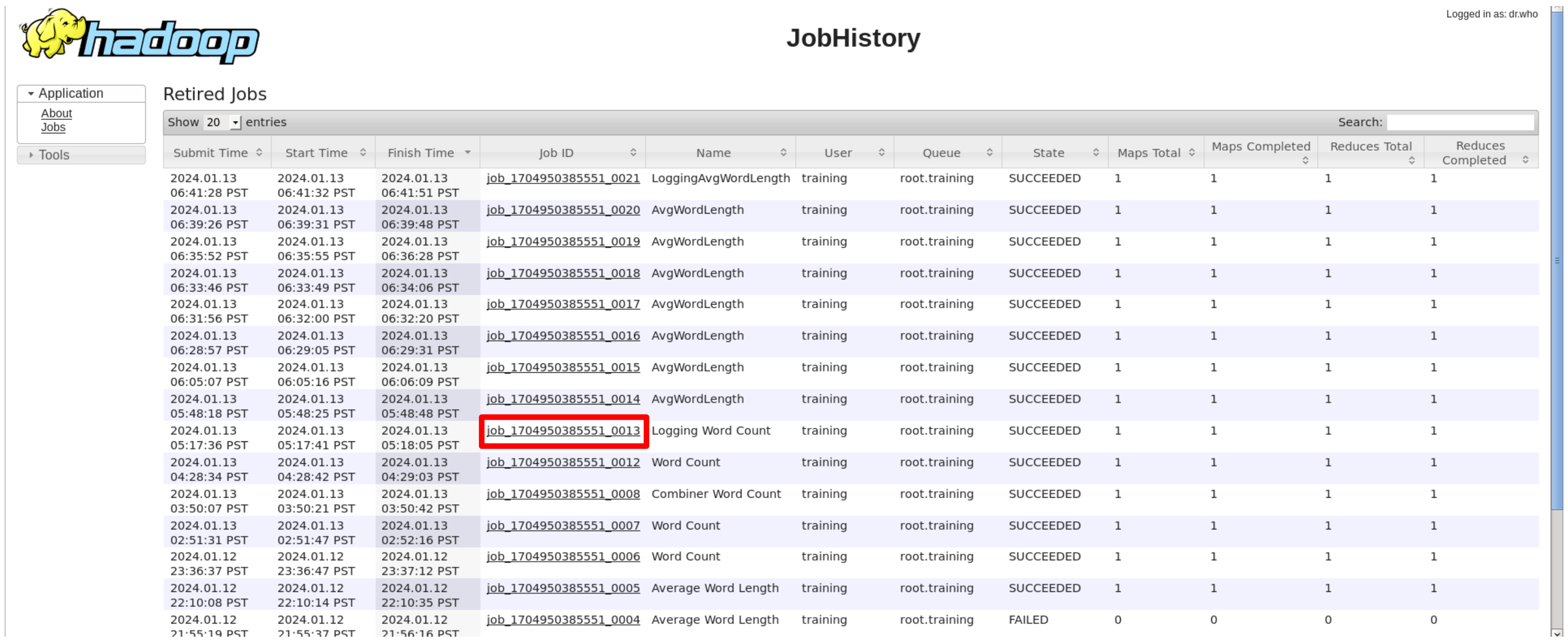
```
e.tar.gz lgwc_res
Programmer: Pei-Wei Tsai
Student ID: 92144979
24/01/13 05:17:35 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0
:8032
24/01/13 05:17:35 WARN mapreduce.JobResourceUploader: Hadoop command-line option
 parsing not performed. Implement the Tool interface and execute your applicatio
n with ToolRunner to remedy this.
24/01/13 05:17:35 INFO input.FileInputFormat: Total input paths to process : 1
24/01/13 05:17:36 INFO mapreduce.JobSubmitter: number of splits:1
24/01/13 05:17:36 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_17
04950385551_0013
24/01/13 05:17:36 INFO impl.YarnClientImpl: Submitted application application_17
04950385551_0013
24/01/13 05:17:36 INFO mapreduce.Job: The url to track the job: http://quickstar
t.cloudera:8088/proxy/application_1704950385551_0013/
24/01/13 05:17:36 INFO mapreduce.Job: Running job: job_1704950385551_0013
24/01/13 05:17:42 INFO mapreduce.Job: Job job_1704950385551_0013 running in uber
 mode : false
24/01/13 05:17:42 INFO mapreduce.Job:  map 0% reduce 0%
24/01/13 05:17:58 INFO mapreduce.Job:  map 100% reduce 0%
24/01/13 05:18:06 INFO mapreduce.Job:  map 100% reduce 100%
24/01/13 05:18:08 INFO mapreduce.Job: Job job_1704950385551_0013 completed succe
ssfully
```

LOGFILE VIA WEB UI

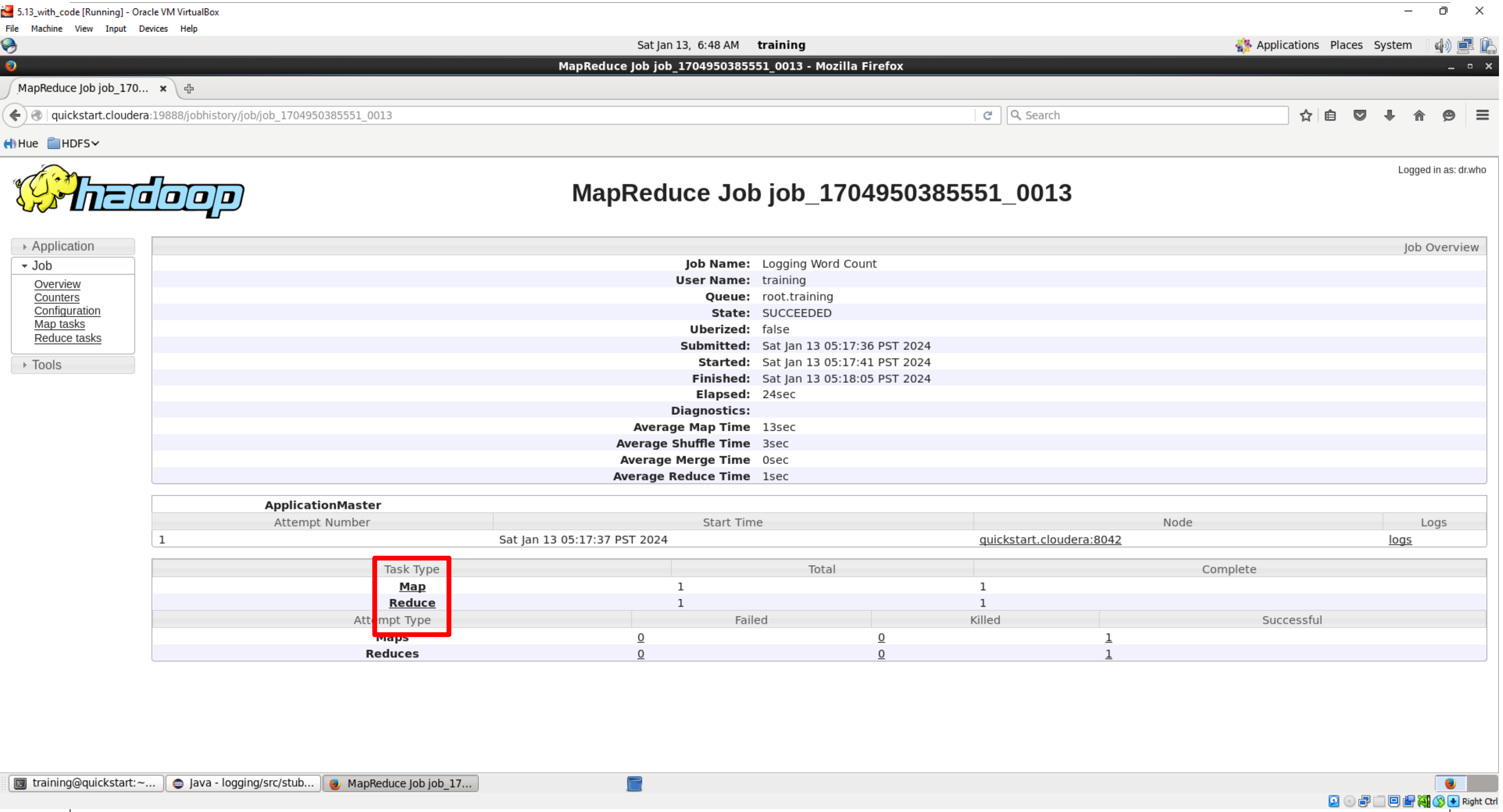


Submit Time	Start Time	Finish Time	Job ID	Name	User	Queue	State	Maps Total	Maps Completed	Reduces Total	Reduces Completed
2024.01.13 05:17:36 PST	2024.01.13 05:17:41 PST	2024.01.13 05:18:05 PST	job_1704950385551_0013	Logging Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 04:28:34 PST	2024.01.13 04:28:42 PST	2024.01.13 04:29:03 PST	job_1704950385551_0012	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 03:50:07 PST	2024.01.13 03:50:21 PST	2024.01.13 03:50:42 PST	job_1704950385551_0008	Combiner Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 02:51:31 PST	2024.01.13 02:51:47 PST	2024.01.13 02:52:16 PST	job_1704950385551_0007	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 23:36:37 PST	2024.01.12 23:36:47 PST	2024.01.12 23:37:12 PST	job_1704950385551_0006	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 22:10:08 PST	2024.01.12 22:10:14 PST	2024.01.12 22:10:35 PST	job_1704950385551_0005	Average Word Length	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 21:55:19 PST	2024.01.12 21:55:37 PST	2024.01.12 21:56:16 PST	job_1704950385551_0004	Average Word Length	training	root.training	FAILED	0	0	0	0
2024.01.10 21:42:11 PST	2024.01.10 21:42:37 PST	2024.01.10 21:43:02 PST	job_1704950385551_0001	Word Count	training	root.training	SUCCEEDED	1	1	1	1

Find the Job in Web UI



Submit Time	Start Time	Finish Time	Job ID	Name	User	Queue	State	Maps Total	Maps Completed	Reduces Total	Reduces Completed
2024.01.13 06:41:28 PST	2024.01.13 06:41:32 PST	2024.01.13 06:41:51 PST	job_1704950385551_0021	LoggingAvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:39:26 PST	2024.01.13 06:39:31 PST	2024.01.13 06:39:48 PST	job_1704950385551_0020	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:35:52 PST	2024.01.13 06:35:55 PST	2024.01.13 06:36:28 PST	job_1704950385551_0019	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:33:46 PST	2024.01.13 06:33:49 PST	2024.01.13 06:34:06 PST	job_1704950385551_0018	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:31:56 PST	2024.01.13 06:32:00 PST	2024.01.13 06:32:20 PST	job_1704950385551_0017	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:28:57 PST	2024.01.13 06:29:05 PST	2024.01.13 06:29:31 PST	job_1704950385551_0016	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 06:05:07 PST	2024.01.13 06:05:16 PST	2024.01.13 06:06:09 PST	job_1704950385551_0015	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 05:48:18 PST	2024.01.13 05:48:25 PST	2024.01.13 05:48:48 PST	job_1704950385551_0014	AvgWordLength	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 05:17:36 PST	2024.01.13 05:17:41 PST	2024.01.13 05:18:05 PST	$\boxed{\text{job\_1704950385551\_0013}}$	Logging Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 04:28:34 PST	2024.01.13 04:28:42 PST	2024.01.13 04:29:03 PST	job_1704950385551_0012	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 03:50:07 PST	2024.01.13 03:50:21 PST	2024.01.13 03:50:42 PST	job_1704950385551_0008	Combiner Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.13 02:51:31 PST	2024.01.13 02:51:47 PST	2024.01.13 02:52:16 PST	job_1704950385551_0007	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 23:36:37 PST	2024.01.12 23:36:47 PST	2024.01.12 23:37:12 PST	job_1704950385551_0006	Word Count	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 22:10:08 PST	2024.01.12 22:10:14 PST	2024.01.12 22:10:35 PST	job_1704950385551_0005	Average Word Length	training	root.training	SUCCEEDED	1	1	1	1
2024.01.12 21:55:19 PST	2024.01.12 21:55:37 PST	2024.01.12 21:56:16 PST	job_1704950385551_0004	Average Word Length	training	root.training	FAILED	0	0	0	0



Find the Job in Web UI

Find the Job in Web UI

Logged in as: dr.who

Map Tasks for job_1704950385551_0013

Show 20 entries Search:

Task					Successful Attempt		
Name	State	Start Time	Finish Time	Elapsed Time	Start Time	Finish Time	Elapsed Time
task_1704950385551_0013_m_000000	SUCCEEDED	Sat Jan 13 05:17:43 -0800 2024	Sat Jan 13 05:17:57 -0800 2024	13sec	Sat Jan 13 05:17:43 -0800 2024	Sat Jan 13 05:17:57 -0800 2024	13sec
ID	State	Start Time	Finish Time	Elapsed Time	Start Time	Finish Time	Elapsed Time

Showing 1 to 1 of 1 entries First Previous 1 Next Last

Show 20 entries Search:

Attempt	State	Status	Node	Logs	Start Time	Finish Time	Elapsed Time	Note
attempt_1704950385551_0013_m_000000_0	SUCCEEDED	map	/default-rack/quickstart.cloudera:8042	logs	Sat Jan 13 05:17:43 -0800 2024	Sat Jan 13 05:17:57 -0800 2024	13sec	
Attempt	State	Status	Node	Logs	Start Time	Finish Time	Elapsed Time	Note

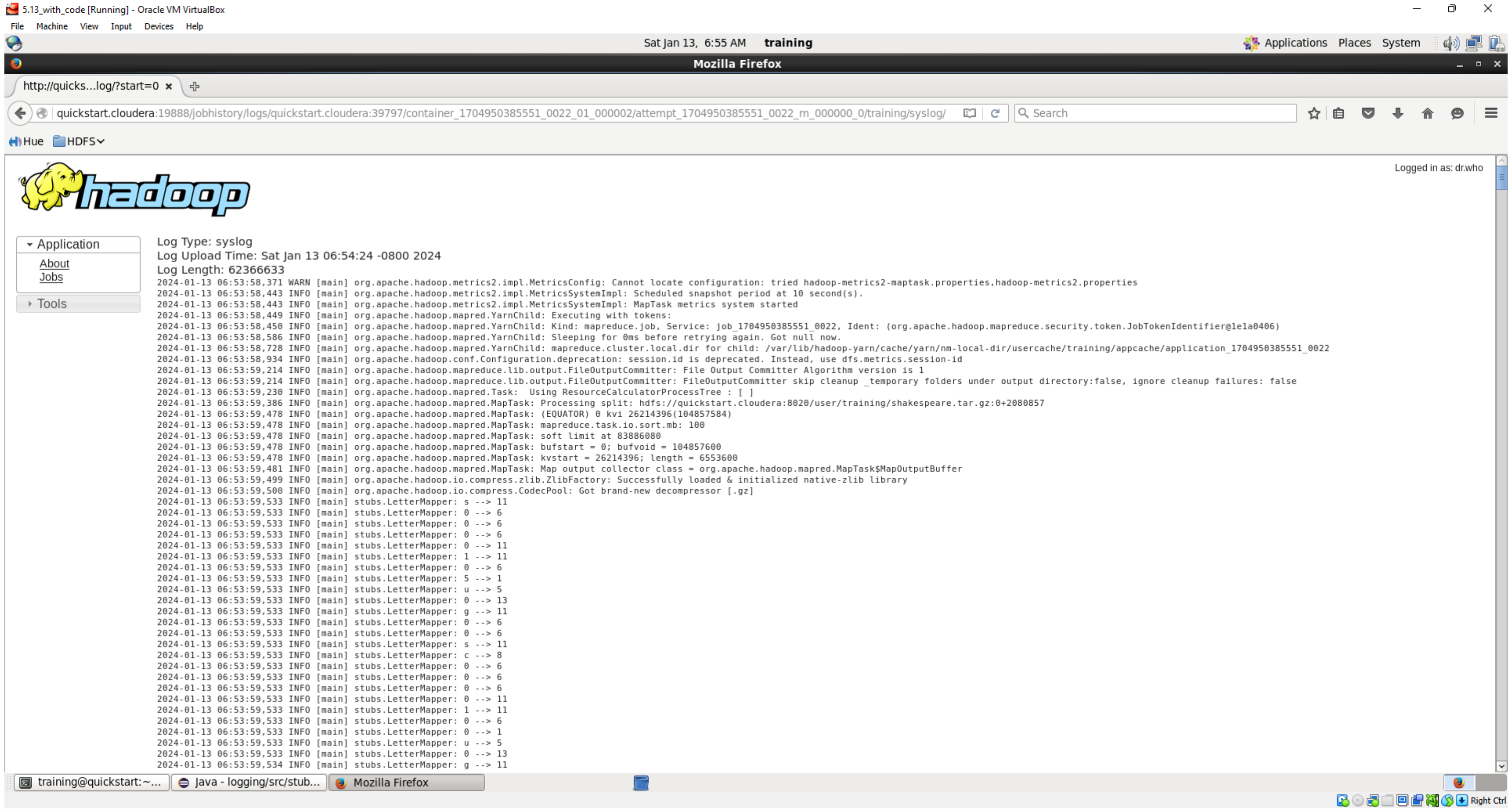
Showing 1 to 1 of 1 entries First Previous 1 Next Last

Find the Job in Web UI

- Click the “Logs” to view the log file.

Find the Job in Web UI

- Mapper Log File



Find the Job in Web UI

- **Reducer Log File**

Log Type: stderr
Log Upload Time: Sat Jan 13 06:54:24 -0800 2024
Log Length: 0

Log Type: stdout
Log Upload Time: Sat Jan 13 06:54:24 -0800 2024
Log Length: 0

Log Type: syslog
Log Upload Time: Sat Jan 13 06:54:24 -0800 2024
Log Length: 78107366
Showing 4096 bytes of 78107366 total. Click here for the full log.

```
--> 76.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 80.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 86.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 93.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 95.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 101.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 105.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 109.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 116.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 120.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 127.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 134.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 138.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 142.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 146.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 150.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 154.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 161.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 165.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 169.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 173.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 177.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 183.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 187.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 191.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 197.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 201.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 205.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 208.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 212.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 216.0
2024-01-13 06:54:13,053 WARN [main] stubs.AverageReducer: z --(Acc.)--> 220.0
```

Log Output Restrictions

If you suspect the input data of being faulty, you may be tempted to log the (key, value) pairs your Mapper receives.

- Reasonable for small amounts of input data.
- If your job runs across 500GB of input data, you could be writing up to 500GB of log files!
- Remember to think at scale when do so.

Instead of wrap vulnerable sections of code in *try{…}* blocks, write logs in the *catch {…}* blocks helps you only log down the critical data.

RETRIEVING JOB INFORMATION WITH COUNTERS

What are Counters?

Counters provide a way for Mappers or Reducers to pass aggregate values back to the driver after the job has completed.

- Their values are also visible from the JobTracker’s Web UI.
- They are reported on the console when the job ends.

Very basic: just have a name and a value.

- Value can be incremented within the code.

What are Counters?

Counters are collected into Groups.

- Within the group, each Counter has a name.

Example: A group of counters called RecordType

- Names: TypeA, TypeB, TypeC
- Appropriate Counter can be incremented as each record is read in the Mapper.

Counters

Counters can be set and incremented via the method.

- context.getCounter(*group, name*).increment(*amount*);

Example

- context.getCounter(“RecordType”,”A”).increment(1);

What are Counters?

To retrieve Counters in the Driver code after the job is complete, use code such as examples below in the driver.

```
long typeARecords = job.getCounters().findCounter(“RecordType”,”A”).getValue();

long typeBRecords = job.getCounters().findCounter(“RecordType”,”B”).getValue();
```

Important!

Do NOT rely on a counter's value from the Web UI while a job is running.

- Due to possible speculative execution, a counter's value could appear larger than the actual final value.
- Modifications to counters from subsequently killed/failed tasks will be removed from the final count.

REUSE OF OBJECTS

Reuse of Objects

- It is generally good practice to reuse objects instead of creating many new objects.
- Creating new objects increases the usage of RAM space and overhead of memory allocation.

```
context.write(new Text(word.substring(0, 1)), new DoubleWritable(word.length()));
```

```java
public class LetterMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {

  private static final Logger LOGGER = LogManager.getLogger(LetterMapper.class.getName());

  private final static DoubleWritable mapperValue = new DoubleWritable();
  private Text mapperKey = new Text();

  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {

        String tmp =  context.getConfiguration().get("caseSensitive");
        Boolean caseSensitivie = (tmp.equalsIgnoreCase("true")? true : false);

        String line = (caseSensitivie? value.toString() : value.toString().toLowerCase());
```

The Alternative

- A better practice is to create those frequently used but the value doesn’t need to be kept for long objects outside of the method.

```java
context.write(new Text(word.substring(0, 1)), new DoubleWritable(word.length()));
```

```java
mapperValue.set(word.length());
mapperKey.set(word.substring(0, 1));
context.write(mapperKey, mapperValue);
```

The Alternative

- A better practice is to create those frequently used but the value doesn’t need to be kept for long objects outside of the method.

```java
public class AverageReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

  private static final Logger LOGGER = LogManager.getLogger(AverageReducer.class.getName());

  private final static DoubleWritable reducerValue = new DoubleWritable();

  @Override
  public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
      throws IOException, InterruptedException {

        double wordCount = 0;
        double cnt = 0;
        //LOGGER.setLevel(Level.WARN);

        for (DoubleWritable value : values) {

            wordCount += value.get();
            cnt += 1.0;

            LOGGER.warn(key + " --(Acc.)--> " + wordCount);
        }
        reducerValue.set((wordCount/cnt));

        context.write(key, reducerValue);

  }
}
```

The Alternative

- A better practice is to create those frequently used but the value doesn't need to be kept for long objects outside of the method.

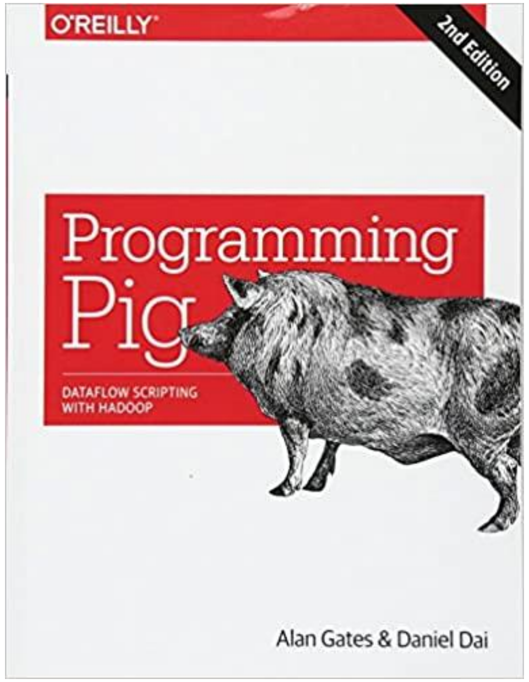
MAP-ONLY JOB

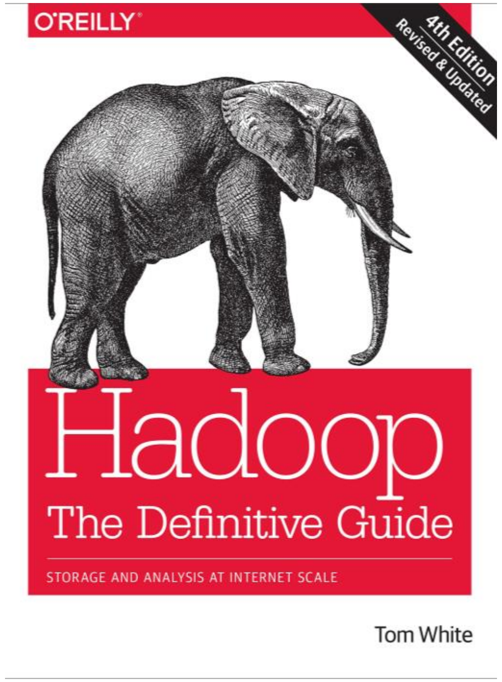
Map-Only MapReduce Jobs

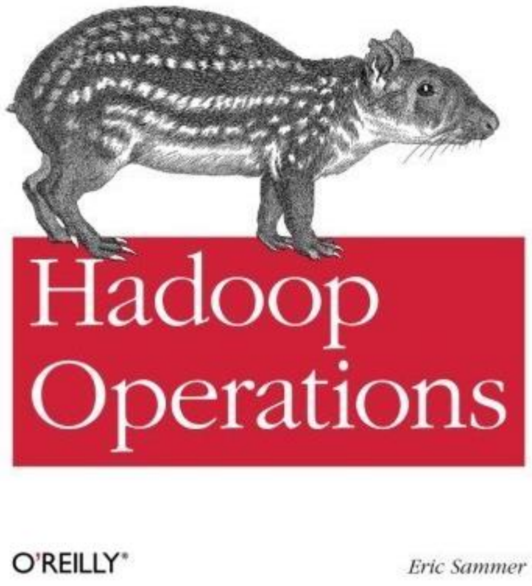
- Many type of jobs, in practical, actually fit in the case that only Mapper is needed.
- Examples
  - File format conversion
  - Input data sampling
  - Image processing
  - Extract, Transform, Load (ETL) data processing methods

Map-Only MapReduce Jobs

- To create a Map-only job, set the number of Reducers to 0 in the Driver code by *job.setNumReduceTasks(0);*
- Call the *job.setOutputKeyClass* and *job.setOutputValueClass* methods to specify the output types.
- Anything written using the *context.write* method in the Mapper will be written to HDFS.
  - The original output should be written in the intermediate data.
  - One file per Mapper will be written.







Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
  - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed.., Sebastopol, California: O'Reilly, 2015.
  - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
  - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017

Unit Summary

- MapReduce Code Explanation.